GUIA DE ESTUDO · CONTROLE DE VERSÃO

Git e GitHub para Iniciantes

Do zero ao trabalho em equipe, em linguagem simples e com exemplos do dia a dia.

10 CAPÍTULOS · DIAGRAMAS · TESTE-SE · 10 EXERCÍCIOS · GLOSSÁRIO · FOLHA DE COLA

```
# seu primeiro commit
git add .
git commit -m "Primeira versão"
git push
```


1 O Básico do Básico
2 Preparando o Ambiente
3 Seu Primeiro Repositório
4 Voltando no Tempo e Desfazendo Erros
5 Branches
6 GitHub
7 Trabalhando em Equipe
8 Boas Práticas e Recursos Avançados
9 Segurança e Socorro
10 Praticando

Sumário

Clique em um capítulo para ir direto a ele.

1 **O Básico do Básico** 4
O que é controle de versão, Git e GitHub

2 **Preparando o Ambiente** 7
Instalar, configurar e criar sua conta

3 **Seu Primeiro Repositório** 11
init, add, commit, log e .gitignore

4 **Voltando no Tempo e Desfazendo Erros** 15
restore, reset, revert e stash

5 **Branches: Trabalhando em Ramos** 18
Branches, merge e conflitos

6 **GitHub: Levando o Código para a Nuvem** 22
push, pull, clone e README

7 **Trabalhando em Equipe: Fork e Pull Request** 26
Fork, PR, revisão e Issues

8 **Boas Práticas e Recursos Avançados** 30
Commits, tags, rebase, Actions e Pages

9 **Segurança e Socorro** 34
Segredos, reflog e situações de aperto

10 **Praticando: Exercícios, Erros Comuns e Glossário** 38
Exercícios, glossário e plano de estudo

APÊNDICES

A **Folha de Cola** 43
Os comandos principais em poucas páginas

B **Gabarito do Teste-se** 46
Respostas das 27 perguntas dos capítulos 1 a 9

Como usar este material

Feito para quem nunca usou Git: toda palavra técnica é explicada na primeira vez que aparece.

A trilha de aprendizado

Fundamentos
Capítulos 1 e 2

Primeiros comandos
Capítulos 3 e 4

Ramos e nuvem
Capítulos 5 e 6

Colaboração
Capítulo 7

Nível avançado
Capítulos 8 e 9

Prática
Capítulo 10

O que você vai encontrar em cada capítulo

NESTE CAPÍTULO VOCÊ VAI

- Os objetivos do capítulo, no começo de cada um

DICA

Atalhos e truques que facilitam a vida.

ATENÇÃO

Pegadinhas e erros comuns. Vale ler duas vezes.

NOTA

Explicações extras e curiosidades que ajudam a entender.

PARA GUARDAR

A ideia principal que você deve levar do assunto.

TESTE-SE · perguntas para responder sem olhar. O gabarito fica no Apêndice B.

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] No fim de cada capítulo, para você conferir o que aprendeu

Como praticar

Os comandos aparecem em blocos de código, e os resultados esperados em blocos de texto logo abaixo. Para acompanhar, use uma **pasta de testes** no seu computador: você pode errar à vontade, porque nada do que fizer ali estraga um projeto de verdade. O projeto de exemplo é a `clinica`, um sistema de marcação de exames, mas qualquer pasta com alguns arquivos serve.

DICA DE ESTUDO

Digite os comandos em vez de copiar e colar. Errar e corrigir é parte do aprendizado, e com o Git quase tudo tem volta.

CAPÍTULO 1 · FUNDAMENTOS

1 O Básico do Básico

NESTE CAPÍTULO VOCÊ VAI

- Entender o problema que o Git resolve e por que todo programador o usa
- Diferenciar Git (o programa) de GitHub (o site)
- Conhecer o vocabulário básico: repositório, commit, branch e remoto

Antes de qualquer comando, vamos entender **para que** isso serve. Este capítulo não tem nada para digitar: é a base para tudo que vem depois.

O problema das mil versões

Você já viu uma pasta assim?

```
trabalho_final.docx
trabalho_final_v2.docx
trabalho_final_v2_corrigido.docx
trabalho_final_AGORA_VAI.docx
trabalho_final_AGORA_VAI_mesmo.docx
```

Com um trabalho individual, já é confuso. Em um projeto de programação, com **dezenas de arquivos** e **várias pessoas** mexendo ao mesmo tempo, vira um pesadelo: quem alterou o quê? Qual é a versão certa? Como voltar ao ponto em que tudo funcionava, antes daquela mudança que quebrou o sistema?

O que é controle de versão

Controle de versão é um sistema que **registra o histórico** de todas as mudanças em um projeto, para que você possa ver, comparar e **voltar no tempo**.

Uma analogia: os **pontos de salvamento** de um videogame. Antes de enfrentar o chefão, você salva o jogo. Se der errado, volta ao ponto salvo em vez de recomeçar do zero. O controle de versão faz isso com o seu código, e ainda guarda quem salvou, quando e por quê.

O que é Git

O **Git** é o programa de controle de versão mais usado do mundo. Foi criado em 2005 por Linus Torvalds, o criador do Linux. Suas características principais:

- **Roda no seu computador**, e funciona sem internet.
- **Guarda o histórico completo** do projeto dentro dele, em uma pasta escondida chamada `.git`.

- É **gratuito e de código aberto**.
- Funciona com qualquer tipo de arquivo de texto, mas é feito sob medida para código.

O que é GitHub

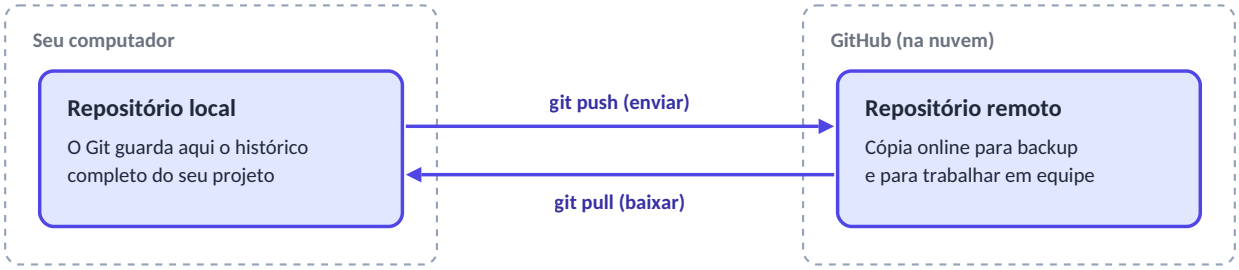
O **GitHub** é um **site** (um serviço na nuvem) que hospeda projetos Git e oferece ferramentas para trabalhar em equipe: revisão de código, lista de tarefas, automações e muito mais. Pertence à Microsoft.

Uma analogia: o Git é como um **editor de fotos** instalado no seu computador, e o GitHub é como uma **rede social para código**, onde você publica o que fez, guarda uma cópia segura e colabora com outras pessoas.

	Git	GitHub
O que é	Um programa	Um site e serviço online
Onde fica	No seu computador	Na nuvem
Precisa de internet?	Não	Sim
Serve para	Guardar o histórico do projeto	Hospedar projetos e colaborar
Dá para usar sem o outro?	Sim	Não: ele é feito para projetos Git

Existem outros sites parecidos com o GitHub, como **GitLab** e **Bitbucket**. Todos usam o mesmo Git por baixo, então o que você aprende aqui vale para eles também.

Git fica no seu computador; o GitHub guarda uma cópia na nuvem



O Git funciona sem internet. O GitHub só entra quando você quer enviar ou baixar mudanças.

Figura 1.1 · O histórico vive no seu computador. O GitHub guarda uma cópia e permite trocar mudanças com outras pessoas.

O vocabulário que você vai ouvir

Palavra	Significado simples	Analogia
Repositório (repo)	A pasta do projeto com todo o seu histórico	O jogo com todos os saves
Commit	Um registro das mudanças, com uma mensagem explicando o que mudou	Um ponto de salvamento
Branch (ramo)	Uma linha paralela de trabalho	Um rascunho separado da versão oficial
Merge	Juntar um ramo de volta ao principal	Colar o rascunho no documento final
Remoto (remote)	A cópia do repositório em outro lugar, como o GitHub	O backup na nuvem
Clone	Baixar uma cópia de um repositório	Fotocopiar o projeto inteiro
Push	Enviar seus commits para o remoto	Subir os saves para a nuvem
Pull	Baixar do remoto as mudanças novas	Atualizar seu jogo com os saves da equipe
Pull Request	Um pedido para que sua mudança seja aceita no projeto	"Revisa e aceita meu trabalho?"

Não precisa decorar: cada termo aparece de novo nos capítulos seguintes, com exemplos.

Por que vale a pena aprender

- **Voltar no tempo:** desfazer um erro sem desespero.
- **Experimentar sem medo:** testar ideias em um ramo separado e descartar se não der certo.
- **Trabalhar em equipe:** várias pessoas no mesmo projeto, sem sobrescrever o trabalho umas das outras.
- **Portfólio:** seu perfil no GitHub mostra o que você sabe fazer. Recrutadores olham.
- **Exigência do mercado:** praticamente toda vaga de tecnologia pede Git.

PARA GUARDAR NESTE CAPÍTULO

- O **Git** guarda o histórico do projeto no seu computador; o **GitHub** é o site que hospeda uma cópia online.
- Um **commit** é um ponto de salvamento, e um **branch** é uma linha paralela de trabalho.
- **Push** envia, **pull** baixa e **clone** copia um projeto inteiro.

TESTE-SE · responda sem olhar. O gabarito está no final do material.

1. Qual é a diferença entre Git e GitHub?
2. O que é um commit?
3. Por que o Git é melhor do que copiar a pasta e chamar de "v2", "v3"?

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] Sei dizer o que o controle de versão resolve
- [ ] Sei a diferença entre Git e GitHub
- [ ] Sei o que são repositório, commit e branch
- [ ] Sei o que fazem push, pull e clone

CAPÍTULO 2 · FUNDAMENTOS

2 Preparando o Ambiente

NESTE CAPÍTULO VOCÊ VAI

- Instalar o Git e configurar seu nome e e-mail
- Aprender os poucos comandos de terminal de que você precisa
- Criar sua conta no GitHub e conectar o computador a ela

Para praticar, você precisa de três coisas: o Git instalado, uma conta no GitHub e um pouquinho de intimidade com o terminal. Vamos por partes.

Instalando o Git

No Windows: baixe o instalador em `git-scm.com` e siga o assistente deixando as opções padrão. Ele instala também o **Git Bash**, um terminal que entende os comandos usados neste material.

No Mac: abra o Terminal e digite `xcode-select --install`, ou use o Homebrew com `brew install git`.

No Linux (Ubuntu ou Debian):

```
sudo apt update
sudo apt install git
```

Para confirmar que deu certo, abra o terminal (no Windows, o Git Bash) e digite:

```
git --version
```

Deve aparecer algo como `git version 2.45.0`. O número pode ser diferente: o importante é aparecer uma versão.

O terminal em cinco comandos

O **terminal** é uma janela de texto onde você controla o computador digitando comandos, em vez de clicar. O Git é usado principalmente por ele. Você só precisa destes cinco para começar:

Comando	O que faz	Exemplo
`pwd`	Mostra em qual pasta você está agora	`pwd`
`ls`	Lista os arquivos da pasta	`ls`
`cd`	Entra em uma pasta	`cd Documentos`
`cd ..`	Volta uma pasta	`cd ..`
`mkdir`	Cria uma pasta nova	`mkdir projetos`

> **DICA**
> Aperte a tecla `Tab` para o terminal completar nomes de pastas e arquivos. Aperte a seta para cima para repetir o comando anterior. Isso economiza muita digitação.

Dizendo ao Git quem é você

Cada commit leva o nome e o e-mail de quem o fez. Configure uma vez só:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
git config --global init.defaultBranch main
```

- Use **o mesmo e-mail da sua conta do GitHub**, para os commits aparecerem no seu perfil.
- O terceiro comando define `main` como o nome do ramo principal dos novos projetos.
- Para conferir tudo: `git config --list`.

Um editor de código

Você vai editar arquivos o tempo todo. O **VS Code** (gratuito, em `code.visualstudio.com`) é a escolha mais comum e tem o Git integrado. Se quiser que o Git abra o VS Code quando precisar de um editor:

```
git config --global core.editor "code --wait"
```

Criando a conta no GitHub

1. Acesse `github.com` e clique em **Sign up**.
2. Escolha o **nome de usuário** com carinho: ele vira o endereço do seu portfólio (`github.com/seunome`). Prefira algo profissional e simples.
3. Confirme o e-mail.
4. Ative a **verificação em duas etapas (2FA)** em *Settings*, *Password and authentication*. Ela protege a conta mesmo que alguém descubra sua senha.

> **IMPORTANTE**
> Guarde os códigos de recuperação da 2FA em um lugar seguro. Se perder o celular sem eles, você pode perder o acesso à conta.

Como o Git prova que você é você

Para enviar código ao GitHub, o Git precisa se identificar. **A senha da sua conta não funciona mais para isso.** Há três caminhos:

Caminho	Como funciona	Quando escolher
Credential Manager (HTTPS)	Na primeira vez abre o navegador para você entrar, e guarda o login	Mais fácil no Windows, que já o instala com o Git
GitHub CLI	Você instala o programa `gh` e roda `gh auth login`	Simples em qualquer sistema
Chave SSH	Você cria um par de "chaves" e cadastra a pública no GitHub	Muito usado por quem programa no dia a dia

Para quem quer a chave SSH:

```
ssh-keygen -t ed25519 -C "seu@email.com"
# aperte Enter nas perguntas para aceitar o padrão
cat ~/.ssh/id_ed25519.pub
```

O último comando mostra a **chave pública**. Copie o texto, vá ao GitHub em *Settings*, *SSH and GPG keys*, *New SSH key* e cole. Teste a conexão com:

```
ssh -T git@github.com
```

Se aparecer uma mensagem com o seu nome de usuário, está tudo certo. **Nunca compartilhe o arquivo sem `.pub`** (a chave privada): ele é como a senha da sua conta.

Se algo der errado

Mensagem	O que costuma ser
`git: command not found`	O Git não está instalado ou o terminal foi aberto antes da instalação. Feche e abra de novo
`Permission denied (publickey)`	A chave SSH não foi cadastrada no GitHub, ou foi cadastrada a errada
`fatal: not a git repository`	Você está em uma pasta que ainda não é um repositório (veremos no capítulo 3)
`Please tell me who you are`	Falta configurar `user.name` e `user.email`

PARA GUARDAR NESTE CAPÍTULO

- Confirme a instalação com `git --version` e configure `user.name` e `user.email` uma vez.
- Cinco comandos de terminal bastam para começar: `pwd`, `ls`, `cd`, `cd ..` e `mkdir`.
- Ative a 2FA no GitHub e use Credential Manager, GitHub CLI ou chave SSH, nunca a senha.

TESTE-SE · responda sem olhar. O gabarito está no final do material.

1. Qual comando confirma que o Git está instalado?
2. Por que configurar `user.name` e `user.email`?
3. Por que ativar a verificação em duas etapas no GitHub?

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] Instalei o Git e `git --version` funciona
- [ ] Configurei meu nome, e-mail e o ramo `main`
- [ ] Criei minha conta no GitHub e ativei a 2FA
- [ ] Conectei meu computador ao GitHub (Credential Manager, `gh` ou SSH)

3 Seu Primeiro Repositório

CAPÍTULO 3 · PRIMEIROS COMANDOS

NESTE CAPÍTULO VOCÊ VAI

- Criar seu primeiro repositório com `git init`
- Usar o ciclo `status`, `add` e `commit` e ler o histórico com `log`
- Impedir que arquivos sensíveis entrem no Git com o `.gitignore`

Hora de colocar a mão na massa. Vamos versionar um projeto de exemplo chamado `clinica`: pode ser o sistema de marcação de exames que montamos antes ou qualquer pasta com alguns arquivos.

Passo 1: criar o repositório

```
mkdir clinica
cd clinica
git init
```

```
Initialized empty Git repository in /home/voce/clinica/.git/
```

O `git init` ("iniciar") cria uma pasta escondida chamada `.git`. É o "cérebro" do repositório: nela o Git guarda todo o histórico. Você nunca mexe nela à mão. Se apagá-la, a pasta volta a ser comum e **o histórico se perde**.

As três áreas do Git

Esta é a ideia mais importante do capítulo. Um arquivo passa por **três lugares** antes de entrar para o histórico:

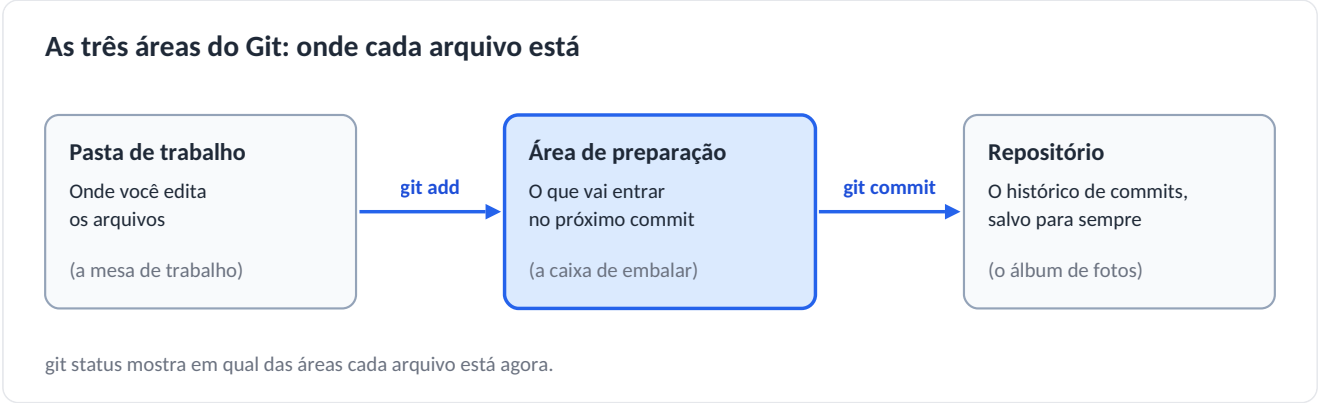


Figura 3.1 · Você edita na pasta de trabalho, escolhe o que entra com `git add` e registra com `git commit`.

Área	O que é	Analogia
Pasta de trabalho	Os arquivos como você os vê e edita	A mesa de trabalho
Área de preparação (staging)	A lista do que vai entrar no próximo commit	A caixa onde você embala o que vai despachar
Repositório	O histórico de commits guardados	O álbum de fotos

Por que existe a caixa do meio? Porque ela deixa você **escolher** o que entra em cada commit. Se mexeu em cinco arquivos mas só três tratam da mesma tarefa, você embala esses três em um commit e os outros dois em outro.

Passo 2: o primeiro arquivo e o `git status`

Crie um arquivo `README.md` (pelo editor ou pelo terminal):

```
echo "# Clínica: marcação de exames" > README.md
git status
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
```

O `git status` é o seu **painel de controle**: mostra o que mudou e em qual área cada arquivo está. Use-o o tempo todo. Aqui, o arquivo é **untracked** (não rastreado): o Git sabe que ele existe, mas ainda não o acompanha.

Passo 3: `git add`, colocar na caixa

```
git add README.md
git status
```

```
Changes to be committed:
        new file:   README.md
```

Agora o arquivo está na **área de preparação**, pronto para o commit. Para adicionar tudo o que mudou de uma vez, use `git add .` (o ponto significa "tudo da pasta atual").

Passo 4: `git commit`, tirar a foto

```
git commit -m "Adiciona README inicial"
```

```
[main (root-commit) 3f2a9c1] Adiciona README inicial
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

O texto entre aspas depois do `-m` é a **mensagem do commit**: uma frase curta dizendo **o que mudou**. O `3f2a9c1` é o **hash**, uma espécie de impressão digital única desse commit.

Passo 5: `git log`, ver o histórico

```
git log --oneline
```

```
3f2a9c1 Adiciona README inicial
```

Com mais commits, a lista cresce, do mais recente para o mais antigo:

```
a81d5e7 Cria rota de agendamentos
c4b0912 Configura conexão com o banco
3f2a9c1 Adiciona README inicial
```

Sem o `--oneline`, o `git log` mostra também o autor, a data e a mensagem completa de cada commit.

O ciclo do dia a dia

Quase todo o trabalho com Git é repetir estes passos:

Passo	Comando	Para quê
1. Editar	(seu editor)	Fazer a mudança
2. Conferir	`git status` e `git diff`	Ver o que mudou
3. Preparar	`git add arquivo`	Escolher o que entra
4. Registrar	`git commit -m "mensagem"`	Salvar no histórico

O `git diff` mostra **linha por linha** o que você alterou e ainda não preparou. Depois do `add`, use `git diff --staged` para ver o que está na caixa.

O `.gitignore`: o que o Git deve ignorar

Alguns arquivos **nunca** devem ir para o repositório: senhas, bibliotecas baixadas, arquivos temporários. O arquivo `.gitignore` lista o que o Git deve fingir que não existe. Crie-o na raiz do projeto:

```
# bibliotecas baixadas (o npm install recria)
node_modules/

# senhas e configurações locais
.env

# arquivos de log e do sistema
*.log
.DS_Store
```

> **ATENÇÃO**
>
> O arquivo `.env` guarda a senha do banco de dados (como no projeto de marcação de exames). Se ele for para o GitHub, qualquer pessoa pode ver. Crie o `.gitignore` **antes** do primeiro `git add .`. Se um arquivo já foi commitado, ignorá-lo depois não o remove do histórico (veremos como lidar com isso no capítulo 9).

```
git add .gitignore
git commit -m "Adiciona .gitignore"
```

Boas mensagens de commit, em resumo

- Curtas e claras: **"Adiciona login"** é melhor que "mudanças".
- Dizem **o que** o commit faz, no presente: "Corrige erro no cadastro".
- **Um commit, uma ideia.** Evite um commit gigante com dez assuntos misturados.

No capítulo 8 voltamos a isso com um padrão usado por equipes.

PARA GUARDAR NESTE CAPÍTULO

- `git init` cria o repositório; a pasta `.git` guarda o histórico.
- O arquivo passa por três áreas: pasta de trabalho, área de preparação (`git add`) e repositório (`git commit`).
- Use `git status` o tempo todo e crie o `.gitignore` antes do primeiro `add .` .

TESTE-SE · responda sem olhar. O gabarito está no final do material.

1. O que o `git init` cria?
2. Qual a diferença entre `git add` e `git commit` ?
3. Para que serve o `.gitignore` ?

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] Criei um repositório com `git init`
- [ ] Fiz meu primeiro commit e vi o resultado com `git log --oneline`
- [ ] Sei descrever as três áreas do Git
- [ ] Criei um `.gitignore` com `.env` e `node_modules/`

CAPÍTULO 4 · PRIMEIROS COMANDOS

4 Voltando no Tempo e Desfazendo Erros

NESTE CAPÍTULO VOCÊ VAI

- Ver o que mudou e consultar o passado do projeto
- Desfazer alterações e commits sem pânico
- Guardar trabalho pela metade com o `git stash`

A grande vantagem do Git é poder **errar sem medo**: quase tudo tem volta. Este capítulo é o seu kit de primeiros socorros. A regra é saber **qual comando usar em cada situação**.

Três ideias antes dos comandos

- **HEAD** é o "marcador de página" do Git: o ponto onde você está agora, normalmente o seu último commit.
- `HEAD~1` significa "um commit antes do atual", `HEAD~2` "dois antes", e assim por diante.
- Alterações que **nunca foram commitadas** não têm volta se você as descartar. Já o que foi commitado quase sempre pode ser recuperado. Por isso: **commite cedo e com frequência**.

Ver o que mudou e olhar o passado

```
git diff                    # o que mudou e ainda não está na área de preparação
git diff --staged           # o que está na área de preparação
git log --oneline           # a lista de commits
git show a81d5e7            # os detalhes e as mudanças de um commit
```

O `git show` recebe o hash do commit, que você pega no `git log --oneline`. Basta copiar os primeiros 7 caracteres.

A tabela de socorro

Situação	Comando
Mudei um arquivo e quero **descartar** a mudança	`git restore arquivo`
Dei `git add` sem querer e quero **tirar da caixa**	`git restore --staged arquivo`
Errei a mensagem do **último commit** (ainda não enviei)	`git commit --amend -m "nova mensagem"`
Esqueci um arquivo no **último commit**	`git add arquivo` e depois `git commit --amend --no-edit`
Quero **desfazer o último commit**, mantendo as mudanças	`git reset --soft HEAD~1`
Quero desfazer um commit **que já foi enviado**	`git revert a81d5e7`
Preciso **trocar de tarefa** sem commitar	`git stash` e depois `git stash pop`

> **ATENÇÃO**
> `git restore arquivo` descarta a mudança **para sempre**, porque ela nunca foi commitada. Pare e confira com `git diff` antes de rodá-lo.

Reset ou revert?

Os dois desfazem commits, mas de jeitos diferentes. A escolha depende de **o commit já ter sido enviado ao GitHub ou não**:

	`git reset`	`git revert`
O que faz	Volta o histórico para trás, como se o commit nunca tivesse existido	Cria um **novo commit** que desfaz as mudanças do anterior
O histórico	É reescrito	Fica intacto
Quando usar	Só em commits que **ainda não foram enviados**	Em commits **já enviados** ou compartilhados
Risco	Alto, se usado em histórico compartilhado	Baixo

Uma analogia: o `reset` é **apagar a página** do caderno; o `revert` é **escrever uma correção** na página seguinte. Se outras pessoas já leram a página, apagá-la causa confusão.

O `reset` tem três modos:

Comando	Commit	Área de preparação	Pasta de trabalho
`git reset --soft HEAD~1`	Desfeito	**Mantém** as mudanças	Mantém
`git reset HEAD~1` (misto, o padrão)	Desfeito	Esvaziada	Mantém
`git reset --hard HEAD~1`	Desfeito	Esvaziada	**Apaga as mudanças**

> **CUIDADO**
> `git reset --hard` joga fora o trabalho não commitado, sem confirmação. Use só quando tiver certeza. Se estiver em dúvida, prefira `--soft`.

Guardando trabalho pela metade: `git stash`

Você está no meio de uma tarefa quando pedem uma correção urgente. Ainda não dá para commitar, porque está pela metade. O `stash` ("esconder") guarda tudo em uma gaveta e deixa a pasta limpa:

```
git stash          # guarda as mudanças e limpa a pasta
# ...faz a correção urgente, commita...
git stash pop      # traz de volta o que estava guardado
git stash list     # lista o que há na gaveta
```

Olhando um commit antigo

Para ver como o projeto estava em um commit do passado, sem alterar nada:

```
git switch --detach 3f2a9c1
```

O Git avisa que você está em **detached HEAD** ("cabeça solta"). Não se assuste: significa só que você está olhando o passado, fora de um ramo. Para voltar ao presente:

```
git switch main
```

PARA GUARDAR NESTE CAPÍTULO

- `restore` desfaz mudanças de arquivos; `reset` desfaz commits ainda não enviados; `revert` desfaz commits já enviados, sem reescrever o histórico.
- `git reset --hard` e `git restore` apagam trabalho não commitado, sem volta.
- O `git stash` guarda o trabalho pela metade em uma gaveta, e o `git stash pop` o traz de volta.

TESTE-SE · responda sem olhar. O gabarito está no final do material.

1. Qual comando tira um arquivo da área de preparação sem perder a edição?
2. Por que o `revert` é mais seguro que o `reset` em commits já enviados?
3. O que o `git stash` faz?

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] Sei usar `git diff` e `git log --oneline` para ver o que mudou
- [ ] Sei descartar uma mudança e tirar um arquivo da área de preparação
- [ ] Sei a diferença entre `reset` e `revert`
- [ ] Sei guardar e recuperar trabalho com `git stash`

5 Branches: Trabalhando em Ramos

CAPÍTULO 5 · RAMOS E NUVEM

NESTE CAPÍTULO VOCÊ VAI

- Entender os branches (ramos) como universos paralelos do projeto
- Criar, trocar e juntar ramos com `switch` e `merge`
- Resolver um conflito de merge, o grande susto do iniciante

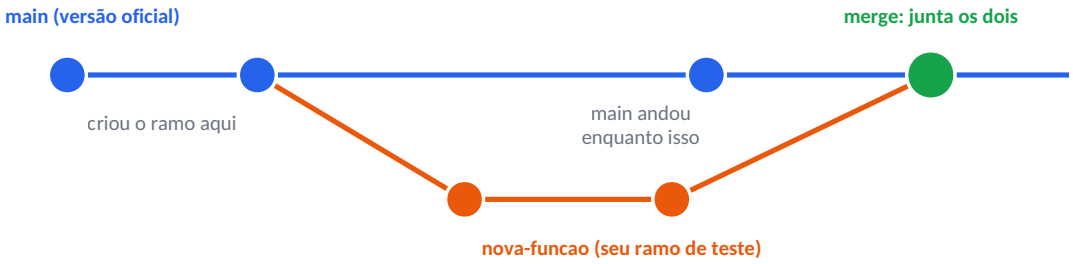
Imagine que você tem a versão do sistema que **funciona** e quer testar uma ideia nova, como um login. E se der errado? Você não quer quebrar o que já funciona. A solução do Git são os **branches**.

O que é um branch

Um **branch** (ramo) é uma **linha paralela de trabalho**. Você sai da versão principal, faz suas experiências em um ramo separado e, quando estiver satisfeito, **junta** de volta. Se não gostar, descarta o ramo e nada acontece com o projeto principal.

Uma analogia: você tem o documento oficial e faz uma **cópia de rascunho** para experimentar. Se o rascunho ficar bom, cola as mudanças no oficial; se não, joga fora.

Branches: um ramo paralelo que depois é juntado ao principal



Cada círculo é um commit. Você testa ideias no ramo sem mexer na versão que já funciona.

Figura 5.1 · O ramo sai da `main`, recebe seus próprios commits e depois é juntado de volta com um merge.

O ramo principal costuma se chamar `main` (em projetos mais antigos, `master`). Ele deve guardar a versão que **funciona**.

Por que usar branches

- **Testar ideias** sem quebrar o que já funciona.
- **Trabalhar em várias tarefas** ao mesmo tempo, cada uma no seu ramo.

- **Em equipe**, cada pessoa trabalha no seu ramo, sem atrapalhar as outras.
- **Branches são baratos:** criá-los é instantâneo e não duplica os arquivos.

Os comandos

Comando	O que faz
`git branch`	Lista os ramos (o atual aparece com `*`)
`git switch -c nome`	**Cria** um ramo novo e já muda para ele
`git switch nome`	Muda para um ramo que já existe
`git merge nome`	Junta o ramo `nome` ao ramo em que você está
`git branch -d nome`	Apaga um ramo que já foi juntado

> **NOTA**
> Em tutoriais mais antigos você verá `git checkout -b nome` e `git checkout nome`. Funcionam igual, mas `switch` é a forma mais nova e mais clara.

Passo a passo: criando e juntando um ramo

Vamos adicionar a página de contato da clínica em um ramo separado.

```
git switch -c feature/contato
```

```
Switched to a new branch 'feature/contato'
```

Edite ou crie arquivos e faça commits normalmente. Eles ficam **só nesse ramo**:

```
echo "Telefone: (21) 0000-0000" > contato.txt
git add contato.txt
git commit -m "Adiciona página de contato"
```

Volte para o ramo principal. Repare que o arquivo `contato.txt` **desaparece** da pasta, porque ele só existe no outro ramo:

```
git switch main
ls
```

Agora junte o ramo à `main`:

```
git merge feature/contato
```

```
Updating 3f2a9c1..b7c0d44
Fast-forward
 contato.txt | 1 +
 1 file changed, 1 insertion(+)
```

Pronto: o `contato.txt` apareceu na `main`. Como o ramo já cumpriu seu papel, apague-o:

```
git branch -d feature/contato
```

Dois tipos de merge

Tipo	Quando acontece	Resultado
Fast-forward	A `main` não andou enquanto você trabalhava no ramo	O Git só "anda a linha para frente", sem criar commit extra
Merge commit	A `main` também recebeu commits nesse tempo	O Git cria um commit de junção, que une as duas linhas

Os dois estão certos: o Git escolhe sozinho.

Conflitos: quando duas mudanças brigam

Um **conflito** acontece quando duas pessoas (ou dois ramos) alteram **a mesma linha** do mesmo arquivo. O Git não sabe qual versão manter e pede que **você** decida. Parece assustador, mas é rotina e tem solução em poucos passos.

Ao rodar o merge, o Git avisa:

```
Auto-merging contato.txt
CONFLICT (content): Merge conflict in contato.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Se você abrir o arquivo, vai ver **marcadores** dentro dele:

```
<<<<<<< HEAD
Telefone: (21) 1111-1111
=======
Telefone: (21) 2222-2222
>>>>>>> feature/contato
```

Marcador	Significado
`<<<<<<< HEAD`	Começa a versão do ramo **em que você está**
`=======`	Separa as duas versões
`>>>>>>> feature/contato`	Termina a versão do ramo que você está **juntando**

Como resolver, passo a passo:

1. Abra o arquivo e **decida** o que fica: uma das versões, a outra ou uma mistura.
2. **Apague os marcadores** (`<<<<<<<` , `=======` e `>>>>>>>`), deixando só o texto final.
3. Salve e rode `git add contato.txt` para avisar que resolveu.
4. Finalize com `git commit` (o Git já sugere uma mensagem).

Se preferir desistir e voltar ao estado de antes do merge: `git merge --abort` .

DICA

O VS Code destaca os conflitos e mostra botões como **Accept Current Change** e **Accept Incoming Change** acima de cada trecho. Não precisa apagar os marcadores na mão.

Nomes de branches

Use nomes curtos e descritivos, sem espaços nem acentos. Uma convenção comum é usar um prefixo:

Prefixo	Para	Exemplo
`feature/`	Nova funcionalidade	`feature/login`
`fix/`	Correção de erro	`fix/horario-duplicado`
`docs/`	Documentação	`docs/readme`

PARA GUARDAR NESTE CAPÍTULO

- Um branch é uma linha paralela de trabalho; a `main` guarda a versão que funciona.
- `git switch -c nome` cria e entra no ramo; `git merge nome` o junta ao ramo atual.
- Conflito acontece quando duas mudanças tocam a mesma linha: escolha o texto, apague os marcadores, `add` e `commit`.

TESTE-SE · responda sem olhar. O gabarito está no final do material.

1. O que é um branch?
2. Qual comando cria um branch e já muda para ele?
3. O que significam os marcadores `<<<<<<<` e `>>>>>>>` dentro de um arquivo?

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] Sei explicar o que é um branch e por que usar
- [ ] Criei um ramo, fiz um commit nele e o juntei à `main`
- [ ] Sei os dois tipos de merge (fast-forward e merge commit)
- [ ] Já resolvi, ao menos uma vez, um conflito de merge

6

CAPÍTULO 6 · RAMOS E NUVEM

GitHub: Levando o Código para a Nuvem

NESTE CAPÍTULO VOCÊ VAI

- Criar um repositório no GitHub e conectá-lo ao seu projeto
- Enviar e baixar mudanças com `push`, `pull` e `fetch`
- Clonar projetos e escrever um README que apresenta seu trabalho

Até agora tudo ficou no seu computador. Agora vamos levar o projeto para o **GitHub**: ganhar um backup na nuvem, poder acessar de qualquer lugar e, mais adiante, colaborar.

Passo 1: criar o repositório no GitHub

1. No GitHub, clique no **+** no topo e em **New repository**.
2. Dê um **nome** (por exemplo `clinica`) e, se quiser, uma descrição.
3. Escolha **Public** (qualquer pessoa vê) ou **Private** (só você e quem você convidar).
4. **Não marque** "Add a README" nem outros arquivos, porque seu projeto local já tem o seu.
5. Clique em **Create repository**.

O GitHub mostra uma página com um endereço (URL) do repositório, parecido com:

```
https://github.com/seunome/clinica.git
git@github.com:seunome/clinica.git
```

A primeira é a versão **HTTPS** e a segunda é a versão **SSH**. Use a que combina com a forma de autenticação que você configurou no capítulo 2.

Passo 2: conectar o projeto local ao GitHub

Dentro da pasta do projeto:

```
git remote add origin https://github.com/seunome/clinica.git
git remote -v
```

- `git remote add origin URL` guarda o endereço do repositório remoto com o apelido **`origin`**. É só um nome: "origin" é a convenção para o repositório principal na nuvem.
- `git remote -v` lista os remotos cadastrados, para você conferir.

Passo 3: enviar com `git push`

```
git push -u origin main
```

Leitura: *"Envie o ramo `main` para o remoto `origin`."* O `-u` faz o Git **lembrar** esse caminho, então nas próximas vezes basta digitar `git push`. Atualize a página do GitHub: seus arquivos e commits já estão lá.



Figura 6.1 · Os comandos que levam as mudanças para frente (add, commit, push) e os que trazem do GitHub (fetch, pull).

Baixando mudanças: `fetch` e `pull`

Se outra pessoa (ou você, em outro computador) enviou commits ao GitHub, o seu repositório local precisa se atualizar:

Comando	O que faz	Quando usar
`git fetch`	**Só baixa** as novidades, sem mexer nos seus arquivos	Quando quer olhar antes de juntar
`git pull`	**Baixa e já junta** no seu ramo (é um `fetch` seguido de `merge`)	No dia a dia, para se atualizar

> **DICA**
> Crie o hábito de dar `git pull` **antes de começar a trabalhar**. Isso evita muitos conflitos, porque você parte da versão mais recente.

Clonando um projeto existente

Para baixar um projeto que já está no GitHub, inclusive o seu em outro computador:

```
git clone https://github.com/seunome/clinica.git
cd clinica
```

O `git clone` cria a pasta, baixa **todos os arquivos e todo o histórico** e já deixa o `origin` configurado. Você não precisa de `git init` nem de `git remote add`.

A rotina do dia a dia, resumida

```
git pull                          # 1. atualiza
# ...trabalha nos arquivos...
git status                        # 2. confere
git add .                         # 3. prepara
git commit -m "Adiciona filtro"   # 4. registra
git push                          # 5. envia
```

O que você encontra no GitHub

Aba	Para que serve
Code	Os arquivos do projeto e o README
Issues	Lista de tarefas, bugs e ideias
Pull requests	Pedidos de mudança para revisar (capítulo 7)
Actions	Automações, como rodar testes a cada envio (capítulo 8)
Settings	Configurações: quem acessa, proteções, site, etc.

Ao clicar em **Commits**, você vê o mesmo histórico do `git log`, mas com autor, data e as mudanças linha por linha. Ao clicar em um arquivo e depois em **History**, você vê só o que aconteceu com ele.

O README: o cartão de visita do projeto

O arquivo `README.md` é a primeira coisa que aparece na página do repositório. Escreva nele **o que o projeto faz**, **como rodar** e **que tecnologias usa**. Ele é escrito em **Markdown**, uma forma simples de formatar texto:

```
# Clínica: marcação de exames

Sistema web para marcar exames, feito com Node.js e MySQL.

## Como rodar
1. Clone o projeto
2. Rode `npm install`
3. Crie o arquivo `.env` (veja `.env.example`)
4. Rode `node server.js`

## Tecnologias
- Node.js e Express
- MySQL
```

Escreva	Vira
`# Título`	Um título grande (`##` e `###` para menores)
`**texto**`	**Negrito**
`- item`	Item de lista
`` `código` ``	Trecho de código
`[texto](endereço)`	Um link

Um bom README é o que mais valoriza um projeto no seu portfólio.

Erros comuns no `push` e no `pull`

Mensagem	O que significa	O que fazer
`rejected (non-fast-forward)`	O GitHub tem commits que você não tem	Rode `git pull` e depois `git push`
`Permission denied` ou `Authentication failed`	Falha de identificação	Revise o capítulo 2 (login, `gh` ou chave SSH)
`remote origin already exists`	Você já tinha cadastrado o `origin`	Veja com `git remote -v` e corrija com `git remote set-url origin URL`

PARA GUARDAR NESTE CAPÍTULO

- `git remote add origin URL` liga o projeto ao GitHub; `git push -u origin main` envia pela primeira vez.
- `git fetch` só baixa; `git pull` baixa e junta. Dê `pull` antes de começar a trabalhar.
- `git clone URL` baixa um projeto inteiro, com todo o histórico, e já o deixa configurado.

TESTE-SE · responda sem olhar. O gabarito está no final do material.

1. O que é o `origin`?
2. Qual a diferença entre `git fetch` e `git pull`?
3. Quando usar `git clone`?

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] Criei um repositório no GitHub e o conectei ao meu projeto
- [ ] Enviei meus commits com `git push` e os vi no site
- [ ] Sei a diferença entre `fetch`, `pull` e `clone`
- [ ] Escrevi um README com o que o projeto faz e como rodar

7 CAPÍTULO 7 · COLABORAÇÃO
Trabalhando em Equipe: Fork e Pull Request

NESTE CAPÍTULO VOCÊ VAI

- Entender o fluxo de trabalho em equipe: branch, Pull Request e revisão
- Usar fork para contribuir com projetos de outras pessoas
- Escrever bons Pull Requests e usar Issues para organizar tarefas

Aqui o Git mostra para que serve de verdade: **muitas pessoas no mesmo projeto**, sem pisar no trabalho umas das outras. O segredo está em um combinado simples, chamado **Pull Request**.

Dois cenários

Cenário	Como é	O que você faz
Equipe com acesso ao repositório	Você é colega de quem criou o projeto e tem permissão para enviar ramos a ele	Cria um **branch** no próprio repositório e abre um Pull Request
Projeto de outra pessoa (código aberto)	O repositório não é seu e você não pode enviar direto para ele	Faz um **fork** (uma cópia sua), trabalha nela e abre um Pull Request

Nos dois casos, a ideia é a mesma: **ninguém mexe direto na `main`**. Toda mudança passa por revisão.

O que é um fork

Um **fork** é uma **cópia de um repositório de outra pessoa** para a sua conta do GitHub, com um clique no botão **Fork**. Analogia: tirar uma **xerox** do projeto. Na sua cópia você pode mudar o que quiser sem afetar o original.

O que é um Pull Request

Um **Pull Request** (PR) é um **pedido**: *"fiz estas mudanças em um ramo; vocês aceitam puxá-las para o projeto?"*. Ele abre uma conversa em que outras pessoas veem **exatamente o que mudou**, comentam, pedem ajustes e, quando aprovam, juntam ao projeto.

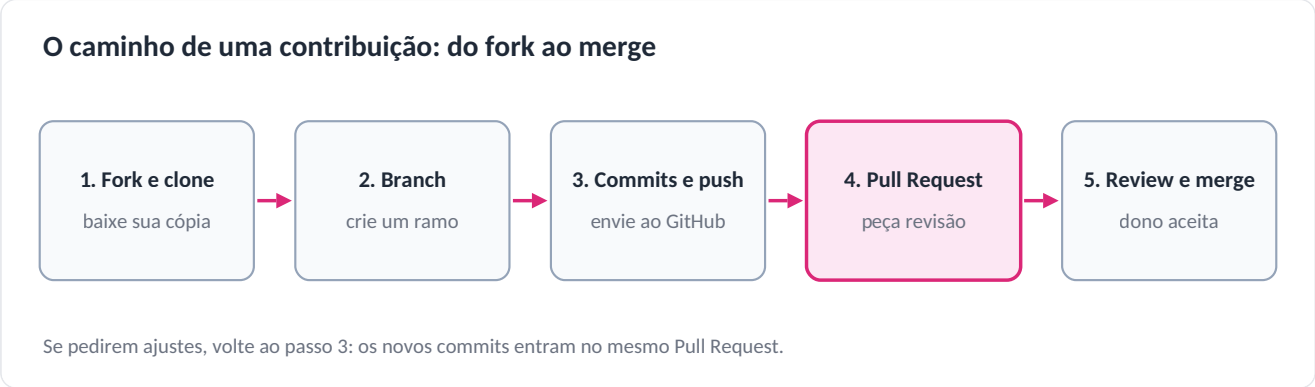


Figura 7.1 · O caminho de uma contribuição, do fork até o merge.

Passo a passo de uma contribuição

1. **Fork:** clique em **Fork** no repositório (só no cenário de projeto alheio).
2. **Clone:** baixe sua cópia com `git clone URL-do-seu-fork`.
3. **Branch:** crie um ramo para a tarefa, nunca trabalhe na `main`: `git switch -c fix/horario-duplicado`.
4. **Commits:** faça as mudanças e commite normalmente.
5. **Push:** envie o ramo para o GitHub: `git push -u origin fix/horario-duplicado`.
6. **Pull Request:** no GitHub, aparece um aviso amarelo com o botão **Compare & pull request**. Clique, escreva um título e uma descrição e confirme.
7. **Revisão:** as pessoas comentam. Se pedirem ajustes, **faça novos commits no mesmo ramo** e dê `push`: o PR se atualiza sozinho.
8. **Merge:** quando aprovado, alguém com permissão clica em **Merge**. Depois, apague o ramo.

Escrevendo um bom Pull Request

Parte	O que colocar	Exemplo
Título	Uma frase clara do que muda	`Corrige agendamento em horário duplicado`
Descrição	O que mudou e por quê, e como testar	"Retorna erro 409 quando o horário já está ocupado"
Link para a tarefa	A Issue que o PR resolve	`Closes #12`

> **DICA**
>
> PRs **pequenos** são revisados mais rápido e com mais atenção. Prefira vários PRs de uma ideia cada a um PR gigante com dez assuntos.

Revisão de código (code review)

A revisão é uma conversa, não uma prova. Quem revisa pode:

- **Comentar** linhas específicas do código.
- **Aprovar** (*Approve*) ou **pedir mudanças** (*Request changes*).

Quem recebe comentários deve agradecer, ajustar o que faz sentido e **explicar com calma** quando discordar. Revisar o código dos outros também é uma ótima forma de aprender.

Issues: a lista de tarefas

Uma **Issue** é uma tarefa, um bug ou uma ideia registrada no repositório. Ela tem título, descrição, comentários, **etiquetas** (*labels*, como `bug` ou `melhoria`) e pode ser atribuída a alguém. Cada Issue recebe um número, como `#12`.

Escrever `Closes #12` (ou `Fixes #12`) na descrição de um PR faz o GitHub **fechar a Issue sozinho** quando o PR for juntado.

As três formas de juntar um PR

Ao clicar em **Merge**, o GitHub oferece opções:

Opção	O que faz	Quando é comum
Merge commit	Mantém todos os commits do ramo e cria um commit de junção	Quando o histórico detalhado importa
Squash and merge	Junta **todos os commits do ramo em um só**	Quando o ramo tem commits bagunçados ("conserta", "agora vai")
Rebase and merge	Reaplica os commits do ramo em linha reta, sem commit de junção	Quando a equipe prefere histórico linear

Cada equipe escolhe a sua. Se estiver em dúvida, **Squash and merge** é uma escolha segura para começar.

Mantendo seu fork atualizado

O projeto original continua evoluindo. Para trazer as novidades para o seu fork, cadastre o original como um segundo remoto, chamado por convenção de `upstream`:

```
git remote add upstream https://github.com/dono-original/clinica.git
git fetch upstream
git switch main
git merge upstream/main
git push
```

O GitHub Flow: o combinado da equipe

Este é o fluxo mais simples e mais usado:

Regra	Por quê
A `main` **sempre funciona**	Qualquer um pode partir dela com segurança
Cada tarefa tem **seu próprio branch**, com nome descritivo	Isola o trabalho
Mudanças entram por **Pull Request**	Ninguém altera a `main` sem revisão
Alguém **revisa** antes do merge	Pega erros cedo e espalha conhecimento
Depois do merge, **apague o branch**	Mantém o repositório limpo

Etiqueta de equipe

- **Dê `git pull` antes de criar um branch novo**, para partir da versão mais recente.
- **Não reescreva histórico compartilhado** (veja `rebase` e `push --force` no capítulo 8).
- **Avise** quando começar algo grande, para a equipe não fazer a mesma coisa ao mesmo tempo.

PARA GUARDAR NESTE CAPÍTULO

- O fluxo é: **branch, commits, push, Pull Request, revisão e merge**. Ninguém mexe direto na `main`.
- Um **fork** é uma cópia sua do projeto de outra pessoa; um **Pull Request** pede que sua mudança seja aceita.
- Escrever `Closes #12` na descrição do PR fecha a Issue automaticamente ao juntar.

TESTE-SE · responda sem olhar. O gabarito está no final do material.

1. O que é um fork?
2. O que é um Pull Request?
3. O que acontece ao escrever `Closes #12` na descrição de um PR?

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] Sei descrever o caminho de uma contribuição, do branch ao merge
- [ ] Sei a diferença entre trabalhar com branch no repositório e com fork
- [ ] Sei escrever o título e a descrição de um bom PR
- [ ] Sei o que são Issues e para que servem as etiquetas

CAPÍTULO 8 · NÍVEL AVANÇADO

8 Boas Práticas e Recursos Avançados

NESTE CAPÍTULO VOCÊ VAI

- Escrever commits no padrão usado por equipes profissionais
- Marcar versões com tags e entender merge, rebase e cherry-pick
- Conhecer o GitHub Actions e o GitHub Pages

Você já domina o essencial. Este capítulo reúne os hábitos e recursos que separam um projeto organizado de uma bagunça. Não precisa decorar: o objetivo é saber que existem e para que servem.

Commits no padrão da equipe

Além de curta e clara, a mensagem pode seguir um padrão chamado **Conventional Commits**: um **prefixo** que diz o tipo de mudança, seguido de uma frase.

Prefixo	Tipo de mudança	Exemplo
`feat:`	Funcionalidade nova	`feat: adiciona cancelamento de exame`
`fix:`	Correção de erro	`fix: impede horário duplicado`
`docs:`	Só documentação	`docs: explica como rodar o projeto`
`refactor:`	Reorganizar código sem mudar o comportamento	`refactor: separa rotas em arquivos`
`test:`	Testes	`test: cobre o cadastro de agendamento`
`chore:`	Tarefas de manutenção	`chore: atualiza dependências`

Ajuda a ler o histórico de relance e permite gerar automaticamente o registro de mudanças de cada versão. Para começar, o que importa é: **commits pequenos, um assunto por commit, mensagem que explica o que e por quê**.

Tags e versões

Uma **tag** é uma **etiqueta fixa** em um commit, geralmente usada para marcar uma **versão** do projeto.

```
git tag v1.0.0                # cria a tag no commit atual
git tag                       # lista as tags
git push origin v1.0.0        # envia a tag ao GitHub
```

No GitHub, a aba **Releases** transforma uma tag em uma página de versão, com notas e arquivos para baixar.

Os números seguem o **versionamento semântico**, `MAJOR.MINOR.PATCH`:

Parte	Quando sobe	Exemplo
MAJOR	Mudança que **quebra** o que existia	`1.4.2` vira `2.0.0`
MINOR	Funcionalidade nova que não quebra nada	`1.4.2` vira `1.5.0`
PATCH	Correção de erro	`1.4.2` vira `1.4.3`

Merge ou rebase?

Os dois trazem as mudanças de um ramo para outro, mas contam a história de jeitos diferentes:

	`git merge`	`git rebase`
O que faz	Junta os dois ramos criando um commit de junção	**Reaplica** seus commits por cima da ponta do outro ramo
Histórico	Mostra que houve ramos paralelos	Fica em **linha reta**
Segurança	Não altera commits existentes	**Reescreve** commits (novos hashes)

```
git switch feature/login
git rebase main          # coloca seus commits depois dos mais novos da main
```

REGRA DE OURO

nunca faça rebase de commits que já foram enviados e que outras pessoas usam. Como o rebase reescreve a história, quem já tinha a versão antiga fica com histórico divergente e conflitos difíceis. Em seus ramos pessoais, ainda não compartilhados, é seguro. Na dúvida, use `merge`.

`cherry-pick`: levar um commit só

O `cherry-pick` ("colher uma cereja") copia **um commit específico** de outro ramo para o atual, sem juntar o ramo inteiro. Útil para levar uma correção urgente:

```
git cherry-pick a81d5e7
```

Investigando o histórico

Comando	O que mostra
`git log --oneline --graph --all`	O histórico em formato de árvore, com todos os ramos
`git log -p arquivo`	Todas as mudanças que um arquivo sofreu
`git blame arquivo`	Quem mexeu em cada linha, e em qual commit
`git show a81d5e7`	Os detalhes de um commit

O `git blame` não é para "achar culpados": serve para descobrir **por que** uma linha está assim, lendo a mensagem do commit.

Para economizar digitação, você pode criar **apelidos** (aliases):

```
git config --global alias.st status
git config --global alias.lg "log --oneline --graph --all"
```

Depois, `git st` roda `git status` e `git lg` mostra a árvore.

GitHub Actions: automações

O **GitHub Actions** roda tarefas **automaticamente** quando algo acontece no repositório, por exemplo **rodar os testes a cada push** ou a cada Pull Request. É a base do que se chama **integração contínua** (CI). As automações ficam em arquivos `.yml` na pasta `.github/workflows/`:

```yaml
name: Testes
on: [push, pull_request]

jobs:
  testar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm test
```

Leitura: *"A cada push ou Pull Request, em uma máquina Linux, baixe o código, instale o Node 20, instale as dependências e rode os testes."* Se algum teste falhar, o PR mostra um **X vermelho**. As versões das actions mudam com o tempo, então confira o número atual na documentação.

GitHub Pages: um site grátis

O **GitHub Pages** publica **sites estáticos** (HTML, CSS e JavaScript) direto de um repositório, de graça. Em *Settings*, *Pages*, escolha o ramo e a pasta, e o GitHub gera o endereço `seunome.github.io/repositorio`. É perfeito para portfólio e páginas de projeto.

Outros recursos para conhecer

Recurso	Para que serve
Branch protection	Impede push direto na `main` e exige PR e revisão
Projects	Quadro de tarefas estilo Kanban
Dependabot	Avisa (e abre PRs) quando uma biblioteca tem versão nova ou falha de segurança
Releases	Páginas de versão com notas e arquivos
Discussions	Fórum para conversas sobre o projeto

PARA GUARDAR NESTE CAPÍTULO

- Prefixos como `feat:` e `fix:` deixam o histórico legível. Marque versões com **tags** e numere por `MAJOR.MINOR.PATCH`.
- `merge` preserva a história; `rebase` a deixa em linha reta, mas **nunca em ramos compartilhados**.
- **Actions** automatiza testes; **Pages** publica um site grátis.

TESTE-SE · responda sem olhar. O gabarito está no final do material.

1. O que significa o prefixo `fix:` em uma mensagem de commit?
2. Por que não fazer rebase em um ramo compartilhado?
3. Para que serve uma tag?

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] Sei escrever mensagens de commit com `feat:`, `fix:` e `docs:`
- [ ] Sei criar e enviar uma tag, e o que significa `MAJOR.MINOR.PATCH`
- [ ] Sei a diferença entre merge e rebase, e quando não usar o rebase
- [ ] Sei para que servem GitHub Actions e GitHub Pages

9

CAPÍTULO 9 · NÍVEL AVANÇADO

Segurança e Socorro

NESTE CAPÍTULO VOCÊ VAI

- Nunca vazar senhas e chaves em um repositório
- Recuperar commits "perdidos" com o `git reflog`
- Ter à mão a solução para as situações de aperto mais comuns

Todo mundo, em algum momento, comete um erro com Git: commitou no ramo errado, enviou o que não devia, apagou algo importante. A boa notícia é que quase tudo tem conserto. A má notícia é uma só: **segredos vazados não têm**. Vamos começar por eles.

Segredos: o erro mais caro

Um **segredo** é qualquer dado que dá acesso a algo: senhas, chaves de API, tokens, o arquivo `.env`, chaves SSH privadas. Se um segredo for enviado ao GitHub, **assuma que foi comprometido**: robôs varrem o GitHub o tempo todo atrás deles, e em poucos minutos alguém pode usá-los.

A analogia: publicar sua senha em um mural público. Apagar o papel depois **não adianta**, porque alguém pode já ter fotografado. No Git é igual, porque o histórico guarda tudo, até o que você apagou em um commit posterior.

Como se prevenir:

- Coloque `.env` e outros arquivos secretos no `.gitignore` **antes** do primeiro commit.
- Crie um arquivo `.env.example`, com os **nomes** das variáveis e **sem os valores**, e este sim vai para o Git. Ele mostra a quem clonar o projeto quais configurações precisa preencher:

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=
DB_PASSWORD=
DB_NAME=clinica
```

- Confira o que vai entrar **antes** de commitar, com `git status` e `git diff --staged`.
- O GitHub tem o **secret scanning**, que avisa quando detecta uma chave conhecida em um repositório.

Se vazou um segredo, nesta ordem:

1. **Revogue ou troque o segredo imediatamente** (mude a senha do banco, gere um novo token). Este é o passo que realmente resolve.
2. Remova do código e use variáveis de ambiente.
3. Só depois, se necessário, limpe o histórico com ferramentas como `git filter-repo` ou o BFG. Isso é trabalhoso e não garante que ninguém já copiou, por isso o passo 1 vem sempre antes.

> **IMPORTANTE**
>
> O mesmo vale para a regra do menor privilégio do guia de MySQL. Se um token vazar, que ele tenha **só as permissões mínimas** e **prazo de validade**, para limitar o estrago.

Protegendo a sua conta

- Ative a **verificação em duas etapas** (2FA).
- Use **chaves SSH** ou o Credential Manager, e **tokens de acesso** com permissões mínimas e prazo de validade.
- Cuidado com **repositórios públicos**: não publique dados pessoais de terceiros (CPF, e-mails, dados de pacientes). A LGPD vale para código também.
- Desconfie de PRs ou de projetos de origem desconhecida que pedem para você rodar comandos estranhos.

O `git reflog`: a rede de segurança

O Git mantém um **diário de por onde o `HEAD` passou**, o **reflog**. Mesmo depois de um `reset --hard` ou de apagar um ramo, o commit costuma continuar existindo por um bom tempo, só "sem nome". O reflog mostra o caminho de volta:

```
git reflog
```

```
a81d5e7 HEAD@{0}: reset: moving to HEAD~2
f09c3b2 HEAD@{1}: commit: Adiciona filtro por exame
b7c0d44 HEAD@{2}: commit: Adiciona página de contato
```

Aqui, o commit `f09c3b2` parece ter "sumido" depois de um reset, mas está no reflog. Para recuperá-lo:

```
git switch -c recuperado f09c3b2     # cria um ramo apontando para ele
```

SOS: situações de aperto e como sair delas

Situação	O que fazer
Commitei na `main` por engano (ainda não dei push)	`git switch -c minha-branch` (o commit vai junto para o novo ramo). Depois `git switch main` e `git reset --hard origin/main` para a `main` voltar ao que estava no GitHub
Fiz push de algo errado	`git revert <hash>` e `git push` : cria um commit que desfaz, sem reescrever a história
Apaguei um branch sem querer	`git reflog` , ache o último commit dele e `git switch -c nome <hash>`
Estou em "detached HEAD"	`git switch main` (se não fez commits ali). Se fez, `git switch -c novo-ramo` antes de sair
Commitei `node_modules/` ou um arquivo que devia ser ignorado	Adicione ao `.gitignore` e rode `git rm -r --cached node_modules` , depois commite
O Git pede a senha toda hora	Configure o Credential Manager, o `gh auth login` ou uma chave SSH (capítulo 2)
Deu conflito no `pull`	Resolva como no capítulo 5: edite, `git add` e `git commit`
Quero desistir de um merge em andamento	`git merge --abort`

O perigo do `push --force`

O `git push --force` **sobrescreve** o histórico do GitHub com o seu. Se outra pessoa tinha commits lá, eles **somem**. Só use em ramos que são só seus, e prefira a versão mais segura:

```
git push --force-with-lease
```

Ela se recusa a sobrescrever se alguém enviou algo novo que você ainda não viu. **Nunca force push na `main` .**

PARA GUARDAR NESTE CAPÍTULO

- Nunca commite senhas, tokens ou `.env` . Se vazar, **revogue o segredo primeiro**: apagar do histórico não basta.
- Use `.env.example` para mostrar quais configurações o projeto precisa, sem os valores.
- O `git reflog` mostra por onde o `HEAD` passou e ajuda a recuperar commits que pareciam perdidos.

TESTE-SE · responda sem olhar. O gabarito está no final do material.

1. Se você enviou uma senha para o GitHub, qual é o primeiro passo?
2. Para que serve o `.env.example`?
3. Qual comando mostra por onde o `HEAD` passou, ajudando a recuperar um commit?

CHECKLIST · marque quando conseguir explicar sem consultar

- [ ] Sei por que apagar um segredo do código não resolve o vazamento
- [ ] Tenho `.env` no `.gitignore` e um `.env.example` nos meus projetos
- [ ] Sei usar o `git reflog` para recuperar um commit
- [ ] Sei por que evitar `push --force`, e que `--force-with-lease` é mais seguro

10 CAPÍTULO 10 · PRÁTICA
Praticando: Exercícios, Erros Comuns e Glossário

NESTE CAPÍTULO VOCÊ VAI

- Resolver 10 exercícios práticos, com respostas
- Reconhecer os erros mais comuns de iniciante
- Consultar o glossário e seguir um plano de estudo de 6 semanas

Git se aprende **fazendo**. Tente resolver cada exercício sozinho, **no terminal**, antes de olhar a resposta. Use uma pasta de testes: aqui, nada do que você fizer pode estragar um projeto de verdade.

Exercícios com respostas

1. Configure seu nome, seu e-mail e o ramo principal padrão como `main`.

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
git config --global init.defaultBranch main
```

2. Crie uma pasta `meu-primeiro-repo`, transforme-a em repositório e faça o primeiro commit com um README.

```
mkdir meu-primeiro-repo
cd meu-primeiro-repo
git init
echo "# Meu primeiro repo" > README.md
git add README.md
git commit -m "Adiciona README inicial"
```

3. Veja o histórico com uma linha por commit.

```
git log --oneline
```

4. Crie um `.gitignore` que ignore `node_modules/` e `.env`, e faça o commit dele.

```
printf "node_modules/\n.env\n" > .gitignore
git add .gitignore
git commit -m "Adiciona .gitignore"
```

5. Edite o README, veja a diferença, descarte a edição e confirme que o arquivo voltou ao normal.

```
echo "linha de teste" >> README.md
git diff
git restore README.md
git status
```

6. Corrija a mensagem do último commit (que ainda não foi enviado).

```
git commit --amend -m "Adiciona .gitignore com node_modules e .env"
```

7. Crie o ramo `feature/contato`, faça um commit nele, volte para a `main`, junte o ramo e apague-o.

```
git switch -c feature/contato
echo "Telefone: (21) 0000-0000" > contato.txt
git add contato.txt
git commit -m "Adiciona página de contato"
git switch main
git merge feature/contato
git branch -d feature/contato
```

8. Provoque e resolva um conflito. Passo a passo:

```
# 1. na main, crie o arquivo e commite
echo "Versão da main" > teste.txt
git add teste.txt
git commit -m "Cria teste.txt"

# 2. em outro ramo, mude a MESMA linha e commite
git switch -c conflito
echo "Versão do ramo" > teste.txt
git commit -am "Muda teste.txt no ramo"

# 3. volte à main e mude a mesma linha de outro jeito
git switch main
echo "Outra versão da main" > teste.txt
git commit -am "Muda teste.txt na main"

# 4. tente juntar: vai dar conflito
git merge conflito

# 5. abra teste.txt, escolha o texto final e apague os marcadores
git add teste.txt
git commit
```

9. Crie um repositório vazio no GitHub chamado `meu-primeiro-repo`, conecte e envie seu projeto.

```
git remote add origin https://github.com/seunome/meu-primeiro-repo.git
git push -u origin main
```

10. Desafio: faça o fluxo completo de um Pull Request. Crie um ramo, envie-o ao GitHub, abra o PR pelo site, junte com "Squash and merge" e atualize sua `main` local.

```
git switch -c feature/rodape
echo "Rodapé da página" > rodape.txt
git add rodape.txt
git commit -m "feat: adiciona rodapé"
git push -u origin feature/rodape
# no GitHub: Compare & pull request, depois Squash and merge

git switch main
git pull
git branch -d feature/rodape
```

Erros comuns de iniciante

Erro	O que acontece	Como evitar
Rodar `git add .` sem olhar o que vai entrar	Arquivos indesejados ou secretos vão para o commit	Confira antes com `git status`
Não ter `.gitignore` antes do primeiro commit	`node_modules/` e `.env` entram no histórico	Crie o `.gitignore` primeiro
Esquecer de commitar e perder trabalho no `restore` ou `reset --hard`	A mudança some, sem volta	Commite cedo e com frequência
Mensagens como "ajustes" e "teste"	O histórico não explica nada	Descreva o que mudou: `fix: impede horário duplicado`
Trabalhar direto na `main`	Fica difícil descartar ou revisar a mudança	Um branch por tarefa
Não dar `git pull` antes de começar	Conflitos desnecessários	Atualize antes de trabalhar
Dar `push --force` em ramo compartilhado	Apaga o trabalho de outras pessoas	Use `--force-with-lease`, e só em ramos seus
Commitar senhas e tokens	Vazamento de segredos	`.gitignore`, `.env.example` e revogar o que vazou
Pânico diante de um conflito	Apagar tudo e recomeçar	Conflito é rotina: leia os marcadores e resolva
Confundir Git com GitHub	Achar que precisa da internet para commitar	O Git é local; o GitHub é só a cópia na nuvem

Glossário: as palavras que você vai ouvir

Palavra	Significado simples
Git	O programa de controle de versão, que roda no seu computador
GitHub	O site que hospeda repositórios Git e oferece ferramentas de equipe
Repositório (repo)	A pasta do projeto, com todo o seu histórico
Commit	Um registro de mudanças com uma mensagem, como um ponto de salvamento
Hash	O código único que identifica cada commit (ex.: `3f2a9c1`)
Staging / área de preparação	Onde ficam os arquivos escolhidos para o próximo commit

Palavra	Significado simples
Branch (ramo)	Uma linha paralela de trabalho
main	O ramo principal, com a versão que funciona
HEAD	O ponto onde você está agora, normalmente o último commit do ramo atual
Merge	Juntar um ramo em outro
Conflito	Quando duas mudanças alteram a mesma linha e o Git pede sua decisão
Rebase	Reaplicar seus commits por cima de outro ramo, deixando o histórico em linha reta
Remote (remoto)	A cópia do repositório em outro lugar, como o GitHub
origin	O apelido padrão do repositório remoto principal
Clone	Baixar uma cópia completa de um repositório
Push	Enviar commits para o remoto
Pull	Baixar do remoto e juntar no seu ramo
Fetch	Baixar do remoto sem juntar
Fork	Uma cópia sua, no GitHub, do repositório de outra pessoa
Pull Request (PR)	Um pedido para que suas mudanças sejam aceitas no projeto
Code review	A revisão do código de um PR por outras pessoas
Issue	Uma tarefa, bug ou ideia registrada no repositório
Tag	Uma etiqueta fixa em um commit, usada para marcar versões
Stash	A gaveta onde você guarda trabalho pela metade
.gitignore	O arquivo que lista o que o Git deve ignorar
README	O arquivo de apresentação do projeto
Reflog	O diário de por onde o HEAD passou, usado para recuperar commits
Actions	O serviço de automações do GitHub (testes, publicação)
2FA	Verificação em duas etapas, que protege a conta

Plano de estudo sugerido

Treinar um pouco **todo dia** vale mais do que estudar muito uma vez por semana. Trinta minutos já ajudam.

Semana	Foco	O que fazer
1	Capítulos 1 a 3	Instalar, configurar e fazer seus primeiros commits. Exercícios 1 a 4
2	Capítulo 4	Praticar desfazer coisas, de propósito. Exercícios 5 e 6
3	Capítulo 5	Branches e conflitos. Exercícios 7 e 8
4	Capítulo 6	Criar repositórios no GitHub e usar `push` e `pull`. Exercício 9
5	Capítulo 7	Fluxo de Pull Request. Exercício 10 e uma contribuição pequena em um projeto
6 em diante	Capítulos 8 e 9	Versione um **projeto seu** de verdade, com README, `.gitignore`, `.env.example` e PRs para você mesmo

Onde praticar mais: o site **Learn Git Branching** (`learngitbranching.js.org`) ensina branches com animações interativas; o **GitHub Skills** (`skills.github.com`) tem cursos curtos dentro do próprio GitHub; o livro **Pro Git** (`git-scm.com/book`) é gratuito, completo e tem versão em português. **Para consultar:** a documentação oficial em `docs.github.com` e `git-scm.com/docs` .

MENSAGEM FINAL

Ninguém decora Git. Profissionais consultam comandos o tempo todo. O que importa é entender **o modelo**: três áreas, commits como pontos de salvamento, branches como rascunhos e Pull Requests como pedidos de revisão. Com isso na cabeça, o resto é consulta.

CHECKLIST FINAL · você concluiu o material quando consegue

- [ ] Criar um repositório, fazer commits e ler o histórico
- [ ] Usar branches e resolver conflitos de merge
- [ ] Enviar um projeto ao GitHub e baixar mudanças
- [ ] Abrir e juntar um Pull Request
- [ ] Explicar por que nunca se commita uma senha, e o que fazer se isso acontecer

A

APÊNDICE A · CONSULTA RÁPIDA

Folha de Cola

Para consultar rápido, sem reler os capítulos.

Configurar (uma vez só)

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
git config --global init.defaultBranch main
```

O ciclo do dia a dia

```
git pull                    # atualizar antes de começar
git status                  # ver o que mudou
git add arquivo             # preparar (ou git add . para tudo)
git commit -m "mensagem"    # registrar
git push                    # enviar ao GitHub
```

Começar um projeto

Quero...	Comando
Transformar a pasta em repositório	`git init`
Baixar um projeto do GitHub	`git clone URL`
Ligar o projeto a um repositório do GitHub	`git remote add origin URL`
Primeiro envio	`git push -u origin main`

Ver o que está acontecendo

Quero...	Comando
Ver o estado dos arquivos	`git status`
Ver o que mudei (não preparado)	`git diff`
Ver o que está na área de preparação	`git diff --staged`
Ver o histórico resumido	`git log --oneline`
Ver o histórico em árvore	`git log --oneline --graph --all`
Ver um commit	`git show hash`
Ver quem mexeu em cada linha	`git blame arquivo`

Branches

Quero...	Comando
Listar ramos	`git branch`
Criar e entrar em um ramo	`git switch -c nome`
Trocar de ramo	`git switch nome`
Juntar um ramo ao atual	`git merge nome`
Apagar um ramo já juntado	`git branch -d nome`
Desistir de um merge com conflito	`git merge --abort`

Desfazer

Situação	Comando
Descartar mudança de um arquivo (**sem volta**)	`git restore arquivo`
Tirar um arquivo da área de preparação	`git restore --staged arquivo`
Corrigir a mensagem do último commit	`git commit --amend -m "nova"`
Desfazer o último commit, mantendo as mudanças	`git reset --soft HEAD~1`
Desfazer um commit já enviado	`git revert hash`
Guardar trabalho pela metade	`git stash` e depois `git stash pop`
Recuperar algo "perdido"	`git reflog`

Remoto e equipe

Quero...	Comando
Só baixar as novidades	`git fetch`
Baixar e juntar	`git pull`
Ver os remotos	`git remote -v`
Atualizar um fork	`git fetch upstream` e `git merge upstream/main`
Marcar uma versão	`git tag v1.0.0` e `git push origin v1.0.0`

O fluxo de um Pull Request

1. Atualize a `main`: `git switch main` e `git pull`.
2. Crie um ramo: `git switch -c feature/nome`.
3. Faça commits pequenos e claros.
4. Envie: `git push -u origin feature/nome`.
5. No GitHub, abra o PR (**Compare & pull request**) e escreva título e descrição.
6. Responda à revisão com novos commits no mesmo ramo.

7. Depois do merge: `git switch main`, `git pull` e `git branch -d feature/nome`.

Lembretes de ouro

Regra	Por quê
Crie o `.gitignore` antes do primeiro commit	`.env` e `node_modules/` não podem entrar
Nunca commite senhas ou tokens	Se vazar, revogue o segredo primeiro
`git status` antes de todo `add` e `commit`	Você vê o que vai entrar
Commits pequenos, um assunto cada	Histórico legível e fácil de desfazer
Um branch por tarefa; nada direto na `main`	Isola e permite revisar
`git pull` antes de começar a trabalhar	Evita conflitos
Nunca `push --force` na `main`	Apaga o trabalho dos outros

B

APÊNDICE B · RESPOSTAS

Gabarito do Teste-se

Confira só depois de tentar responder. Se errou, volte ao capítulo indicado e refaça o exemplo.

Cap.	Pergunta	Resposta
1	Diferença entre Git e GitHub?	O **Git** é o programa que guarda o histórico no seu computador; o **GitHub** é o site que hospeda uma cópia online e oferece ferramentas de equipe
1	O que é um commit?	Um registro das mudanças com uma mensagem, como um **ponto de salvamento** do projeto
1	Por que Git em vez de pastas "v2", "v3"?	Guarda o histórico completo (quem, quando e por quê), permite voltar no tempo e comparar, e evita uma pasta cheia de cópias
2	Comando que confirma a instalação?	`git --version`
2	Por que configurar nome e e-mail?	Cada commit leva o nome e o e-mail de quem o fez; com o mesmo e-mail do GitHub, os commits aparecem no seu perfil
2	Por que ativar a 2FA?	Protege a conta mesmo que alguém descubra a senha
3	O que o `git init` cria?	A pasta escondida `.git`, onde o Git guarda o histórico, transformando a pasta em repositório
3	`git add` x `git commit` ?	O `add` **prepara** (coloca na área de preparação); o `commit` **registra** o que está preparado no histórico
3	Para que serve o `.gitignore` ?	Lista os arquivos que o Git deve ignorar, como `.env` e `node_modules/`
4	Tirar um arquivo da área de preparação?	`git restore --staged arquivo`
4	Por que `revert` é mais seguro que `reset` ?	O `revert` cria um novo commit que desfaz o anterior, **sem reescrever o histórico**; o `reset` o reescreve e atrapalha quem já tem a versão antiga
4	O que faz o `git stash` ?	Guarda as mudanças pela metade em uma "gaveta" e limpa a pasta; o `git stash pop` as traz de volta
5	O que é um branch?	Uma linha paralela de trabalho, usada para testar mudanças sem mexer na versão principal
5	Criar e entrar em um branch?	`git switch -c nome`
5	O que são `<<<<<<<` e `>>>>>>>` ?	**Marcadores de conflito**: delimitam as duas versões da mesma linha, e você deve escolher o texto final e apagá-los
6	O que é o `origin` ?	O apelido padrão do repositório remoto principal (o endereço no GitHub)
6	`git fetch` x `git pull` ?	O `fetch` só baixa as novidades; o `pull` baixa **e já junta** no seu ramo
6	Quando usar `git clone` ?	Para baixar um projeto inteiro do GitHub pela primeira vez, com todo o histórico
7	O que é um fork?	Uma cópia sua, na sua conta do GitHub, do repositório de outra pessoa

Cap.	Pergunta	Resposta
7	O que é um Pull Request?	Um pedido para que as mudanças de um ramo sejam revisadas e aceitas no projeto
7	O que faz `Closes #12`?	Fecha automaticamente a Issue 12 quando o PR for juntado
8	O que é o prefixo `fix:` ?	Indica que o commit é uma **correção de erro**
8	Por que não fazer rebase em ramo compartilhado?	O rebase reescreve commits (novos hashes), e quem já tinha a versão antiga fica com histórico divergente e conflitos
8	Para que serve uma tag?	Marca um commit específico, geralmente uma **versão** do projeto (`v1.0.0`)
9	Enviei uma senha ao GitHub: primeiro passo?	**Revogar ou trocar a senha** imediatamente; apagar do código não basta, pois o histórico guarda tudo
9	Para que serve o `.env.example` ?	Mostra quais variáveis o projeto precisa, **sem os valores**, e pode ir para o Git
9	Qual comando mostra por onde o HEAD passou?	`git reflog`